Kardex Content Tracker

Developer Documentation

Add-on system for the Kardex high-bay warehouse that tracks stored contents via an inventory list and camera images

Version: 1.1
GitHub: https://github.com/enjoyer-c/kardex
Date: 2026-11-02
Author: Malwin Jansson

Table of Contents

1 Purpose and Scope 3
2 System Overview 4
2.1 Hardware 4
2.2 Software Modules 4
3 Workflow 5
3.1 Threads 6
3.2 Reading the Log 7
4 GUI 8
5 Design Decisions 10
6 Setup 13
6.1 Installation 13
6.2 Mounting the SSD 13
6.3 Starting the Program 13
6.4 First Start 14
7 Configuration and Files 15
7.1 Configuration (`config.py`) 15
7.2 Files Managed by the Program 16
7.3 Data on the SSD 17
8 Known Limitations and Ideas 18

1 Purpose and Scope

The Kardex Content Tracker is an add-on for the Kardex high-bay warehouse. The Kardex itself does not record what is stored in each tray. This system closes that gap. When a tray goes back in, the system identifies its tray number and automatically captures its contents. The saved images are displayed in a searchable list, so users can check what is in a tray without ordering it out.

The system works independently of the Kardex: it only uses its own door sensor and cameras and has no connection to the Kardex control. One Raspberry Pi is installed per Kardex unit.

This document is meant for developers who maintain or extend the system. It describes the workflow, the main design decisions and the setup. Usage of the GUI is covered in Section 4.

2 System Overview

2.1 Hardware

Component	Model	Purpose
Controller	Raspberry Pi 4 Model B Rev 1.5	Runs software
Door sensor	Reed contact, GPIO 17	Detects door movement
QR scanner	Raspberry Pi HQ Camera, 16 mm lens	Reads the tray number from the QR code
USB cameras	4x Logitech C920 PRO HD, 1080p, 30 FPS	Capture images of the tray contents
USB hub	Powered USB hub	Supplies power for the USB cameras
Storage	SSD, 128 GB	Stores images and logs

Table 1: Hardware modules and their purpose

2.2 Software Modules

File	Purpose
`main.py`	Flow control: when the door opens, it takes photos with the USB cameras and the QR scanner is asked at the same time, then the photos are saved under the tray number or discarded. Also runs the startup checks.
`gui.py`	Tkinter interface: searchable tray table with image preview and magnifier, error banner, History, Manual Capture and the Camera Setup window.
`config.py`	All constants in one place (paths, GPIO pin, camera and QR settings, tray limits), platform-aware for Windows and Pi.
`door_sensor.py`	Reads the door sensor via GPIO and reports every change. If the GPIO can't be set up, the program still starts and shows an error (a disconnected cable can't be detected).
`qr_code_scanner.py`	Scans the ribbon camera permanently in a background thread (Aruco, then zbar as fallback). At a door opening it looks up which QR code was seen around the door signal.
`camera_streams.py`	Keeps every USB camera open with one background thread per camera, so a capture only decodes the newest frame. Sets fixed camera settings and turns the frames by 180°.
`camera_capture.py`	Takes the newest frame of every camera at the same moment, crops the overlaps, puts the images side by side and saves them under the tray number.
`camera_setup.py`	Finds the connected USB cameras via `/dev/v4l/by-path`, manages their left-to-right order and checks it at startup.
`inventory.py`	Reads, searches and writes the inventory list (crash-safe) and validates tray numbers.
`setup_ribbon_cam.py`	Live preview of the ribbon camera with QR detection for focusing. Started from the Camera Setup window.
`*_mock.py`	Replacements for the hardware modules on Windows, so the program can be tested without hardware.
`run_kardex.sh`	Starts the program and starts it again after a crash or a hanging ribbon camera. Used by the desktop icon and the autostart.

Table 2: Software modules and their purpose

3 Workflow

The program only knows two things: when the door opens or closes and which QR code is visible at the delivery position. The whole logic is one rule, triggered by the door opening. Closing the door does nothing. Nothing is remembered between two door events, so every opening is complete on its own.

All cameras run permanently in the background. When the door opens, the USB photos are taken immediately and, at the same time, the QR scanner is asked which code it has seen around the door signal (from `QR_RECENT_S` before to `QR_SCAN_WINDOW_S` after). In practice the code is usually already known before the door signal arrives. Only then is it decided whether the photos are kept.

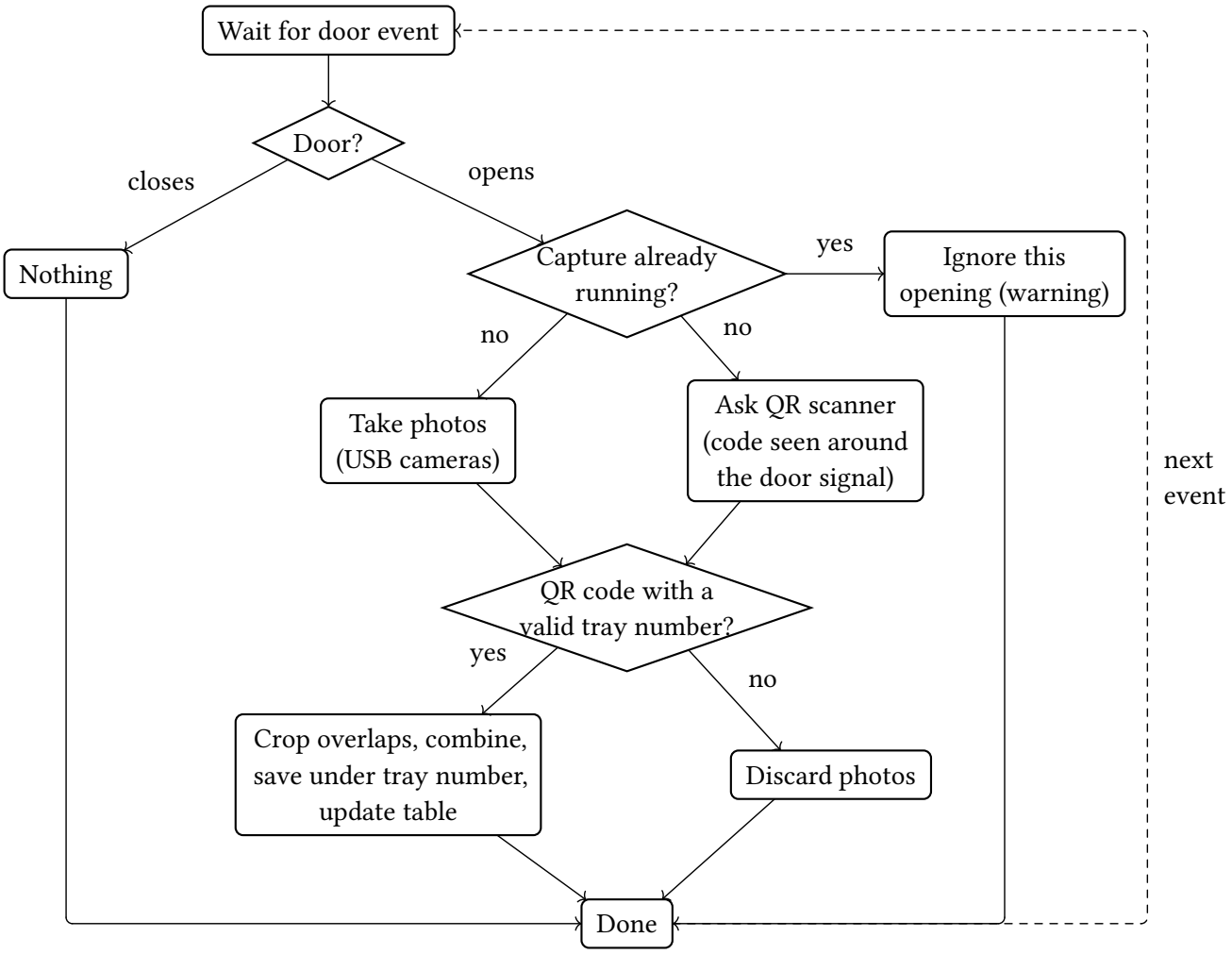


Figure 1: Flow logic: the door opening triggers, the QR code decides.

As Figure 1 shows, the QR code only decides what happens to the photos, never whether they are taken. Photo and tray number come from the same moment, so a photo can never end up under the wrong tray. Table 3 walks through a typical cycle, including two false triggers (e.g. a hand in the light barrier):

Situation	Door	Result
Tray is ordered	opens	Photos taken, no QR code → photos discarded
Tray has moved out	closes	Nothing
Tray is sent back, stands at the front	opens	Photos taken, QR code of tray 12 → saved under tray 12
Tray moves in (a few seconds later)	closes	Nothing
Hand in the light barrier, no tray	opens	No QR code → photos discarded
Hand in the light barrier, tray at front	opens	QR code found → one extra photo of tray 12 is saved

Table 3: Typical cycle with two false triggers

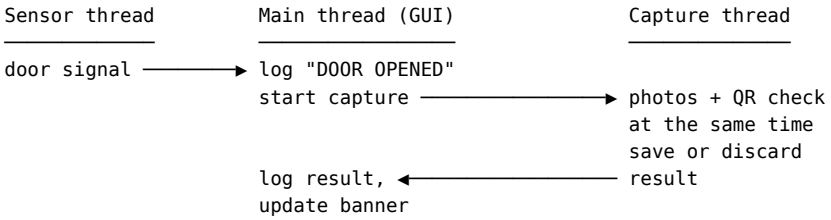
A false trigger cannot break the flow: without a tray at the front it leads nowhere, with a tray at the front it only produces one extra photo. Because only `MAX_IMAGES_PER_TRAY` images are kept per tray, such an extra photo pushes out the oldest one.

Edge cases handled:

- The first sensor report after program start is only logged (“Door state at startup”), it is not a real door movement.
- A door opening while a capture is still running is ignored and logged as a warning. The same applies to a manual capture, which is rejected while another capture is running.
- If the photos fail, the ribbon camera fails or the QR code contains no valid tray number, nothing is saved. The error is shown in the red banner and in the History.
- If the ribbon camera stops delivering frames, a door opening reports an error instead of “no QR code”, and the camera is restarted. If the camera hangs so badly that it can’t be restarted from within the program, the whole program restarts itself (see Section 5).

3.1 Threads

Tkinter is not thread-safe: if two threads change the GUI at the same time, it can freeze or crash at random moments. So only the main thread touches the GUI. Sensor events and capture results are handed over with `root.after()`. The actual capture runs in a background thread, so the GUI never freezes (Listing 1). In addition, every USB camera and the QR scanner have their own permanent background thread (see Section 5), which are not shown here.



Listing 1: One door opening through the threads

3.2 Reading the Log

Every door event is logged with milliseconds, so the timing can be checked afterwards. Listing 2 shows a real cycle from a test at the lift (shortened): first the tray is ordered (no QR code yet, photos discarded), then it is sent back (QR code already seen 0.68 s before the door signal, photos saved).

```
05:49:31.100 [INFO] ===== DOOR OPENED ===== (signal handled after 0 ms)
05:49:33.395 [INFO] Photos taken 0.05 s after the door signal
05:49:33.395 [INFO] No QR code (looked for 2.3 s) -> no tray at the front, photos discarded
05:49:36.673 [INFO]   details: QR code in view: '6' (reader: zbar)
05:49:39.782 [INFO] ===== DOOR CLOSED ===== (signal handled after 0 ms)
05:50:21.022 [INFO] ===== DOOR OPENED ===== (signal handled after 4 ms)
05:50:21.183 [INFO] Photos taken 0.08 s after the door signal
05:50:21.185 [INFO] QR code: tray 6 (seen -0.68 s from the door signal)
05:50:21.185 [INFO] Capture saved for tray 6: finalFrame_20261015_055021.jpg
05:50:24.855 [INFO]   details: QR code '6' out of view
05:50:29.649 [INFO] ===== DOOR CLOSED ===== (signal handled after 0 ms)
```

Listing 2: Log of one cycle: tray 6 ordered and sent back

4 GUI

The program has a single main window (Figure 2). The upper part shows the current state of the system, the lower part the inventory with the latest image of the selected tray. All settings for the cameras are in a separate Camera Setup window (Figure 3).

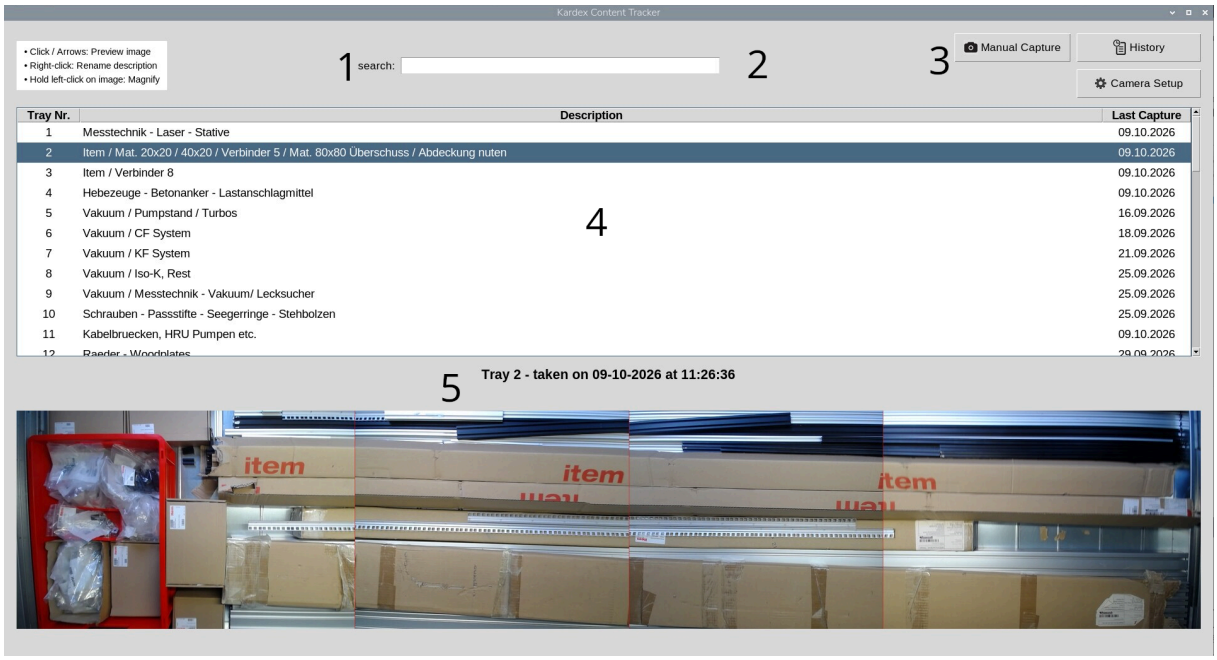


Figure 2: Main GUI overview

No.	Element	Description
1	Search	Filters the table by description while typing.
2	Error banner	Only visible when there is a persistent problem (hidden in the screenshot). Disappears by itself once the problem is solved.
3	Buttons	*Manual Capture*: take photos for a tray number entered by hand (fallback). *History*: log of all events and errors. *Camera Setup*: camera order, overlap per seam and ribbon-cam live preview.
4	Tray table	All trays with description and date of the last capture. Click or arrow keys select a tray; right-click renames its description.
5	Preview	Latest image of the selected tray, scaled to the available space. Hold the left mouse button on the image for a magnifier.

Table 4: Elements of the main window

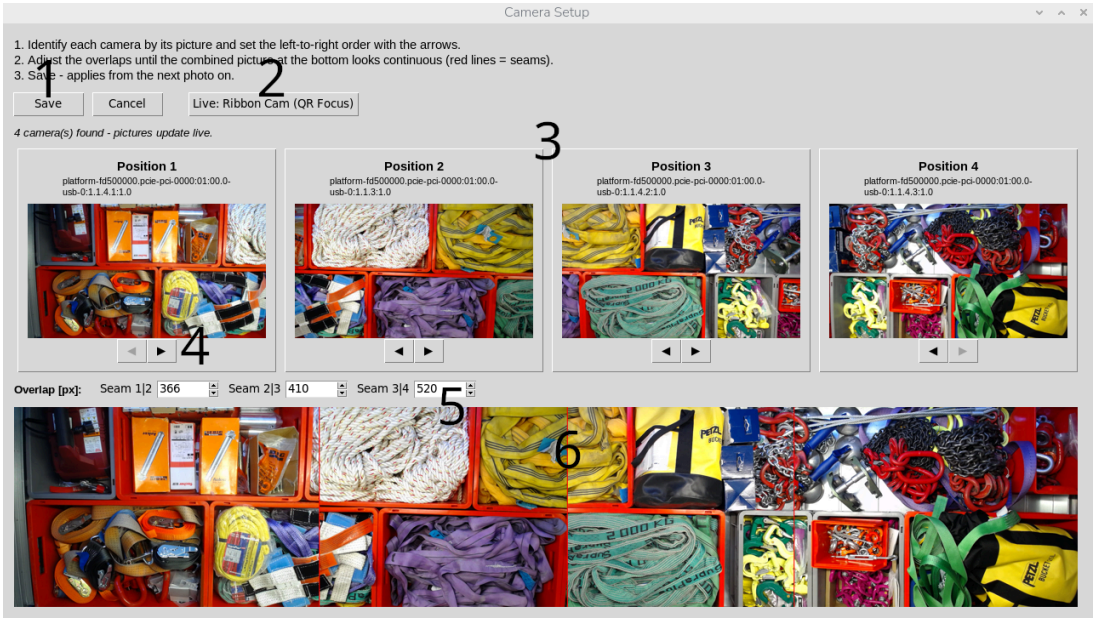


Figure 3: Camera Setup window overview

No.	Element	Description
1	Save button	Saves the camera order and the seam overlaps (`settings.json`) and closes the window. Both are used from the next capture on, no restart needed. *Cancel* closes the window without saving.
2	Live: Ribbon Cam button	Opens a live view of the ribbon camera with QR detection, to align and focus it. While it is open, the program releases the ribbon cam, so no automatic photos are taken (red banner); the camera is taken over again when the view is closed.
3	Camera previews	Live picture of every detected USB camera, updated every 0.5 s, so each camera can be identified by what it sees. The position number shows its place in the left-to-right order.
4	Arrow buttons	Move a camera one position to the left or right until the order matches the physical arrangement.
5	Overlap (px)	One value per seam between two neighboring cameras: how many pixels their pictures overlap. Half of it is cut off each neighbor. Increase it if something appears twice at a seam, decrease it if something is missing. Arrows change it in 2 px steps; a number can also be typed in.
6	Combined preview	The combined picture exactly as it will be saved, with the current overlap values, updated live. The red lines mark the seams and appear only here, not in the saved photos.

Table 5: Elements of the Camera Setup window

5 Design Decisions

One rule: the door opening triggers, the QR code decides

Problem The first version counted door events in a fixed order, so one extra event (e.g. a hand in the light barrier) shifted the whole sequence. The second version scanned the QR code when the door closed and remembered the tray number until it opened again: tray number and photos came from two different moments, and a missed or wrong event in between could save photos under the wrong tray.

Decision When the door opens, the USB photos are taken and the QR scanner is asked at the same time. A valid tray number → the photos are saved under it; no QR code → they are discarded. Closing the door does nothing. Nothing is remembered between two door events.

QR code is scanned permanently, looked up by time

Problem Scanning only for a short moment after the door signal missed codes: one scan on the Pi takes 0.2 to 0.5 s, so only very few frames were checked. In the live preview the code was often never recognized.

Decision A background thread scans every newest frame of the ribbon cam all the time and remembers each code with the time it was seen. At a door opening, the program only looks up which code was seen from `QR_RECENT_S` before to `QR_SCAN_WINDOW_S` after the door signal. In tests, the code was already known 0.7 to 0.9 s before the door opened. A watchdog reports an error instead of "no QR code" if the camera stops delivering frames, and restarts it.

Restart the whole program if the ribbon cam hangs

Problem In the first long-term test the ribbon cam stopped delivering frames after about two hours. The program detected it, but closing and reopening the camera blocked forever inside libcamera. A hanging call in a C library can't be cancelled from Python, so no photos were saved until the program was restarted by hand three days later.

Decision The camera restart runs with a timeout (`QR_CAMERA_RESTART_TIMEOUT_S`). If it hangs or fails, the program logs the reason and exits with `PROGRAM_RESTART_EXIT_CODE`. `run_kardex.sh` then starts it again after 10 s, which reopens all cameras. A normal close of the window is not restarted.

Two QR readers and the still configuration

Problem Labels sit on reflective metal with only a narrow white border around the code. zbar alone could not read some of them. In addition, the ribbon cam ran in the video configuration, with which some codes became unreadable.

Decision OpenCV's Aruco QR reader is tried first (it read the labels on metal reliably), zbar runs as fallback only if Aruco finds nothing. The ribbon cam now uses the still configuration again. The live preview uses exactly the same readers, so what is green there is also read at the door.

All cameras stream permanently

Problem Opening a USB camera, negotiating the format and letting the exposure settle took several seconds; starting the ribbon cam another 1-2 s. The tray starts moving about 2.5 s after the door opens.

Decision Every camera is opened once at program start and keeps delivering frames in the background. A photo only takes the newest frame (USB: `grab()` in the background, `retrieve()` on demand). A camera that stops delivering is reopened automatically; frames older than 1 s are never used.

Fixed settings for the USB cameras

Problem Autofocus and auto white balance ran independently in every camera: each focused on something different and chose its own colors, so the four parts of the picture didn't match. With autofocus, the size of the picture also changed slightly.

Decision Every time a camera is opened, the program sets fixed values via `v4l2-ctl` (`USB_CAMERA_CONTROLS`): autofocus off, focus 0 (infinity), white balance 4000 K, frame rate may not drop in low light. The values were determined once at the lift.

Ribbon cam live preview releases the camera

Problem Two programs can't use the ribbon cam at the same time and the separate live preview for focusing would block the QR scan.

Decision When the preview opens, the program releases its ribbon cam stream and shows a red banner ("no automatic photos"); when the preview closes, the stream is restarted automatically.

Identify USB cameras by port (by-path)

Problem Identical cameras (Logitech C920) report the same or no serial number, so their names collide.

Decision Cameras are identified by `/dev/v4l/by-path`, i.e. by physical USB port. The left-to-right order is set once in the Camera Setup window.

Instead of `by-id` (collides) or `/dev/videoN` (numbering can change after a reboot).

MJPEG and a powered USB hub

Problem Reading several cameras at once caused green artifacts and corrupt images, because the USB bandwidth was exceeded.

Decision The cameras deliver MJPEG (compressed) at a limited frame rate (`USB_CAMERA_FPS`) and are connected via a powered hub. With this, four cameras stream at the same time without problems.

Program data on an SSD, system on the SD card

Problem Frequent writes (images, logs) wear out the SD card.

Decision Captures and logs are written to an SSD mounted at `/mnt/kardex_ssd`; the OS stays on the SD card.

Capture runs in a background thread

Problem Reading the QR code and saving the picture would freeze the window if they ran in the GUI thread, and further door events would only be handled afterwards.

Decision The capture and the camera streams run in their own background threads. Results are handed to the main thread (`root.after`); only the main thread updates widgets.

Python threads don't execute Python code in parallel, but that hardly matters here: the threads mostly wait for the cameras or work inside C libraries (OpenCV, zbar), which release Python's global interpreter lock → the cameras are read and decoded at the same time.

Image preview below the table instead of a pop-up window

Problem The images had to be opened in a separate pop-up window. This window easily end up behind the main window.

Decision The preview sits directly below the tray table and updates on click or arrow keys. It scales to the available space; details are shown with a magnifier.

Crop the overlaps instead of stitching

Problem The OpenCV stitcher failed every time: the cameras hang side by side above objects of different heights, so near and far objects shift by different amounts between two cameras (parallax).

Decision The cameras are mounted rigidly, so the overlap between two neighbors is constant. One overlap value per seam is set once in the Camera Setup window; half of it is cut off each neighbor, then the images are put side by side.

Problems in a red error banner instead of a status display

Problem Errors were only written to the log and the History, so problems like a disconnected camera went unnoticed. A status display (waiting / scanning / capturing) changed too fast to be useful once a capture only took fractions of a second.

Decision Persistent problems (door sensor, ribbon cam, USB cameras, failed scan or capture, open live preview) are shown in a red banner that disappears by itself once the problem is solved. What happened is visible in the History and the "Last Capture" column.

6 Setup

The following steps set up a fresh Raspberry Pi.

6.1 Installation

1. Install Raspberry Pi OS on the SD card and finish the first-boot setup.
2. Check the system time with `timedatectl`. If it is wrong, set it, otherwise `apt` rejects the package lists ("Not live until …"):

```
sudo date -s "YYYY-MM-DD HH:MM"
```

1. Copy the project to the Pi:

```
mkdir -p ~/kardex && cd ~/kardex
git clone https://github.com/enjoyer-c/kardex.git KardexTrackerProgram
```

1. Run the setup script as the normal user (not with `sudo`). It installs the system packages via `apt`, creates the Python environment `.venv` in the project folder and installs OpenCV and pyzbar there in the tested versions. At the end it checks that the Aruco QR reader is available:

```
cd ~/kardex/KardexTrackerProgram
chmod +x setup.sh
./setup.sh
```

6.2 Mounting the SSD

Images and logs are stored on the SSD at `/mnt/kardex_ssd`. It has to be mounted automatically on every boot:

1. Find the UUID of the SSD: `lsblk -f`
2. Create the mount point: `sudo mkdir -p /mnt/kardex_ssd`
3. Add this line to `/etc/fstab` (example for an ext4-formatted SSD):

```
UUID=<your-uuid>  /mnt/kardex_ssd  ext4  defaults,nofail  0  2
```

`nofail` lets the Pi still boot if the SSD is missing.

1. Mount it and give the user write access:

```
sudo mount -a
sudo chown -R $USER:$USER /mnt/kardex_ssd
```

6.3 Starting the Program

From a terminal (error messages are shown directly):

```
cd ~/kardex/KardexTrackerProgram
./run_kardex.sh
```

`run_kardex.sh` starts the program and starts it again after a crash or a hanging ribbon camera. A normal close of the window is not restarted.

For a desktop icon and the app icon in the taskbar, save the following as `kardex.desktop` in `~/.local/share/applications/` (start menu) and on the desktop (`~/Desktop/`). For the autostart after every boot (e.g. after a power failure), copy the same file to `~/.config/autostart/`.

```
[Desktop Entry]
Type=Application
Name=Kardex
Comment=Kardex Shuttle Logistics System
Path=/home/pi/kardex/KardexTrackerProgram
Exec=/home/pi/kardex/KardexTrackerProgram/run_kardex.sh
Icon=/home/pi/kardex/KardexTrackerProgram/icons/app_icon.png
StartupWMClass=Kardex
Terminal=false
Categories=Utility;
```

`StartupWMClass=Kardex` must match the window class of the program, otherwise the taskbar shows a generic icon.

To keep the system log after a reboot (helpful to find the cause of camera problems):

```
sudo mkdir -p /var/log/journal && sudo systemctl restart systemd-journald
```

6.4 First Start

1. Open the door once: `===== DOOR OPENED =====` must appear in the History. If it says CLOSED, flip `DOOR_SENSOR_INVERTED` in `config.py`.
2. Open **Camera Setup**, arrange the cameras in their physical left-to-right order with the arrows, adjust the overlap per seam until the combined picture looks continuous, and click Save. This creates `settings.json`.
3. Use **Live: Ribbon Cam** to align and focus the ribbon camera until the QR code of a tray at the delivery position is shown in green.
4. If other cameras or a different mounting are used: find suitable values for focus and white balance and set them in `USB_CAMERA_CONTROLS` in `config.py`.

7 Configuration and Files

All settings are located in `config.py`. The file is platform-aware, so the same version runs unchanged on Windows (development) and on the Pi. Settings that are changed during operation (camera order, seam overlaps) are not stored in `config.py`, but in `settings.json`, written by the Camera Setup window.

Paths (`BASE_DIR`, `SSD_DIR`, `OUTPUT_DIR`, `LOG_DIR`) and the camera backend (`CAP_BACKEND`) are chosen automatically depending on the platform and normally don't need to be changed.

7.1 Configuration (`config.py`)

USB Cameras

`USB_CAMERA_FOURCC = "MJPG"` Compressed format, needed so several cameras fit on the USB bus.

`USB_CAMERA_RESOLUTION = (1280, 720)` Resolution per camera.

`USB_CAMERA_FPS = 15` Frame rate of the permanent streams.

`USB_CAMERA_MAX_FRAME_AGE_S = 1.0` A frame older than this is never used for a photo.

`USB_CAMERA_RECONNECT_S = 2.0` Pause before a lost or unplugged camera is opened again.

`USB_CAMERA_FIRST_FRAME_TIMEOUT_S = 5.0` Maximum wait for the first frame of a camera that was just plugged in.

`USB_CAMERA_CONTROLS` Fixed camera settings, set via `v4l2-ctl` every time a camera is opened: autofocus off, `focus_absolute = 0` (infinity), auto white balance off, `white_balance_temperature = 4000`, frame rate may not drop in low light. The order matters: autofocus must be switched off before the focus value is set.

`ROTATE_IMAGE_180 = True` The cameras are mounted upside down, every frame is turned before it is used.

`CAMERA_SETUP_PREVIEW_REFRESH_MS = 500` How often the Camera Setup window updates its live pictures.

Picture

`SEAM_OVERLAP_PX = [390, 410, 520]` Overlap in pixels between neighboring cameras, one value per seam. Half of it is cut off each neighbor. Only the default: the values set in the Camera Setup window are stored in `settings.json`.

`MAX_IMAGES_PER_TRAY = 3` Older images of a tray are deleted automatically.

QR scan

`QR_CAPTURE_SIZE = (1332, 990)` Resolution of the ribbon cam stream.

`QR_SCAN_WINDOW_S = 2.0` A QR code counts if it was seen up to this long AFTER the door signal.

`QR_RECENT_S = 1.0` … or up to this long BEFORE the door signal.

`QR_SCAN_GRACE_S = 2.0` Extra time to finish scanning the last frame of the window.

`QR_SCAN_INTERVAL_S = 0.1` Minimum time between two scans. Usually the scan itself takes longer.

`QR_GONE_S = 1.0` "QR code out of view" is only logged after this long without a code, so a single unreadable frame doesn't fill the log.

`QR_FRAME_TIMEOUT_S = 3.0` A frame that takes longer counts as a camera error.

`QR_CAMERA_RESTART_AFTER = 5` After this many camera errors in a row, the ribbon cam is restarted.

`QR_CAMERA_RESTART_TIMEOUT_S = 10.0` A camera restart that takes longer counts as hanging, then the whole program is restarted.

`PROGRAM_RESTART_EXIT_CODE = 75` Exit code that tells `run_kardex.sh` to start the program again.

Door sensor

`DOOR_SENSOR_GPIO = 17` GPIO pin of the door sensor (physical pin 11).
`DOOR_SENSOR_BOUNCE_TIME_MS = 350` Changes within this time are ignored.
`DOOR_SENSOR_INVERTED = True` Which contact state means "door open".
`DOOR_TEST_BUTTON = False` Shows the "TEST: Toggle Door" button (for testing without a magnet).

Trays and GUI

`TRAY_LOWER_LIMIT = 1`, `TRAY_UPPER_LIMIT = 50` Valid tray numbers.
`SEARCH_DEBOUNCE_MS = 300` The search runs this long after the last keystroke.
`PREVIEW_DEBOUNCE_MS = 150` The preview image is loaded this long after the selection stops changing (arrow keys held down).
`LENS_SIZE = 300` Diameter of the magnifier in pixels.
`LENS_ZOOM = 2` Magnification relative to the preview.

System

`LOCK_FILE = /tmp/kardex.lock` Makes sure the program runs only once. The lock is released automatically when the program ends, also after a crash.

7.2 Files Managed by the Program

File	Written by	Content
`inventory.txt`	Rename (GUI)	One description per line, line 1 = tray 1. Written crash-safe.
`settings.json`	Camera Setup	Left-to-right order of the USB cameras (by-path device paths) and the overlap in pixels per seam.

Table 6: Files in the project folder

Both files can also be edited by hand. If no camera order is saved, or a saved camera is no longer connected, all connected cameras are used in USB port order (with a warning and a red banner). If no overlaps are saved, the defaults from `config.py` apply. If the number of overlap values doesn't match the number of cameras, the images are saved side by side uncut and a warning is written to the log.

settings.json

```
{
  "seam_overlap_px": [520, 414, 376],
  "camera_order": [
    "/dev/v4l/by-path/platform-...-usb-0:1.2.4.3:1.0-video-index0",
    "..."
  ]
}
```

7.3 Data on the SSD

Images and logs are written to the SSD, while the operating system stays on the SD card (see Section 5). The storage structure uses the following directories:

```
/mnt/kardex_ssd/
├── captures/
│ ├── 1/
│ │ ├── finalFrame_20260928_095152.jpg
│ │ └── ... (max. MAX_IMAGES_PER_TRAY)
│ ├── 2/
│ └── ... (one folder per tray, 1 to 50)
└── logs/
  ├── kardex.log (current, max. 5 MB)
  └── kardex.log.1 ... .3 (older backups)
```

8 Known Limitations and Ideas

- **System time:** The Pi has no built-in battery-backed clock and gets the time from the network. Without an internet connection, image file names, the "Last Capture" column and the log get wrong timestamps. Since images are sorted by their timestamp, a new image could even be treated as the oldest and deleted.
  Idea: add an RTC module.
- **No check whether the SSD is mounted:** If the SSD is not mounted, images and logs are silently written to the SD card instead, or the program fails at startup.
  Idea: check the mount point at startup and show the problem in the error banner.
- **QR labels:** The labels sit on reflective metal with only a narrow white border around the code. All other tested labels are read reliably by the Aruco reader, only the label of tray 10 is not readable and will be replaced.
  Idea: print new labels with a white border of at least four modules around the code.
- **Parallax at the seams:** The overlap per seam only fits exactly for one height. Objects that are much higher or lower can look slightly doubled or cut off where two images meet. This is a limit of the method, not an error. Stitching, which could handle it in theory, failed (see Section 5).
- **Vibration of the outer right camera:** It vibrates slightly when the door opens or the lift moves, so its part of the picture can be a little blurred.
  Idea: a stiffer camera mount.
- **Door sensor direction and cable:** Which contact state means "open" depends on the sensor (`DOOR_SENSOR_INVERTED`). A disconnected sensor cable can't be detected by software; it reads like a door state.
- **Live preview pauses the automatic mode:** While the ribbon cam live preview is open, the program has released the camera, so no automatic photos are taken (shown in the red banner).
- **Test code still included:** The door test button is switched off (`DOOR_TEST_BUTTON = False`), but its code is still in the program. It can be removed once it is no longer needed for testing without a magnet.
- **Long-term operation not yet tested:** The flow has been tested successfully at the lift. The seam overlaps, the fixed camera settings and the QR detection still have to be checked over a longer period and with differently loaded trays.
- **Ribbon cam hangs:** In the first long-term test the ribbon cam stopped delivering frames once after about two hours. The program now restarts itself in this case, but the cause is still unknown. The system log is now kept after reboots, so the cause can be checked if it happens again.